BIG DATA 201

15.10.25, 9:45 am

MAPREDUCE

WHY MAPREDUCE

A computational/processing component of Hadoop framework meant to process large amounts of data stored on big clusters of cheap commodity machines. Can perform parallel and distributed computations that use potentially large datasets and across a large number of nodes/data nodes. Ideally, it also uses data that is stored locally on a particular node/data node where the job gets executed. Here computations are parallel because there is no communication and dependency between them. They run independent of one another.

- Its architecture makes the code move towards the machines containing the data blocks. This is called **data locality**. This contributes to high performance in MR programming.
- Implements **automatic parallelization**, which means developers need not worry about how the files are processed in parallel, where the splits lie.
- **Scale out** - the number of machines are increased in the Hadoop cluster to support more blocks when the file size increases.

WHAT IS MAPREDUCE

- **Framework**: For writing distributed applications.
- **Purpose**: Process massive datasets (TBs, PBs).
- **Storage**: Uses HDFS in Hadoop clusters.

Process

- **Input Files**: Split into blocks in HDFS.
- **Parallel Processing**: Mapper tasks process blocks simultaneously.
- **Reducer Task**: Consolidates mapper outputs.

Example

- **Scenario**: 100,000 mappers process 100,000 blocks.
- **Time**: 10 minutes if each block takes 10 minutes.

- **Benefit**: High-performance compared to RDBMS single system model.

By default, each block size is 128 MB but can be changed to 256 MB/512 MB by configuring it in hdfs-site.xml during installation of Hadoop.

MapReduce algorithm contains 2 important tasks:

- Map: takes the input data(file blocks),applies the processing logic, converts it into a suitable format.
- Reduce: gets the output of Map task, and consolidates the output of many mapper tasks to produce the final output.

MapReduce programs can be written in Java, C, Python, etc.., but Java is preferred because it :

- supports functional programming with high performance;
- has rich API support for various functional programmatic requirements like string manipulations, data parsing;
- is an open source and a reliable language with good community support;

Resilience / Fault tolerance:
It considers the **replicated copy** of blocks in alternate machines for further processing.
Each node periodically updates its current status via **block report to the master machine**. If a slave node (data node running tasks via Task Tracker slave service) doesn't send a notification, the master (Job tracker) reassigns the current running task of that slave node to other available nodes in the cluster. This makes MapReduce a resilient architecture enabling it to run on cheap commodity servers despite these machine failures.

MAPREDUCE PROGRAMMING MODEL

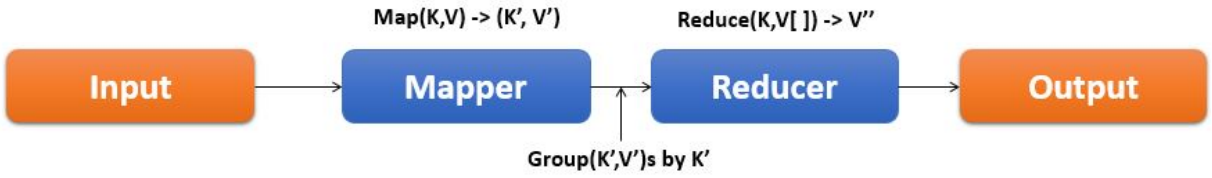
Carried out in 4 phases:

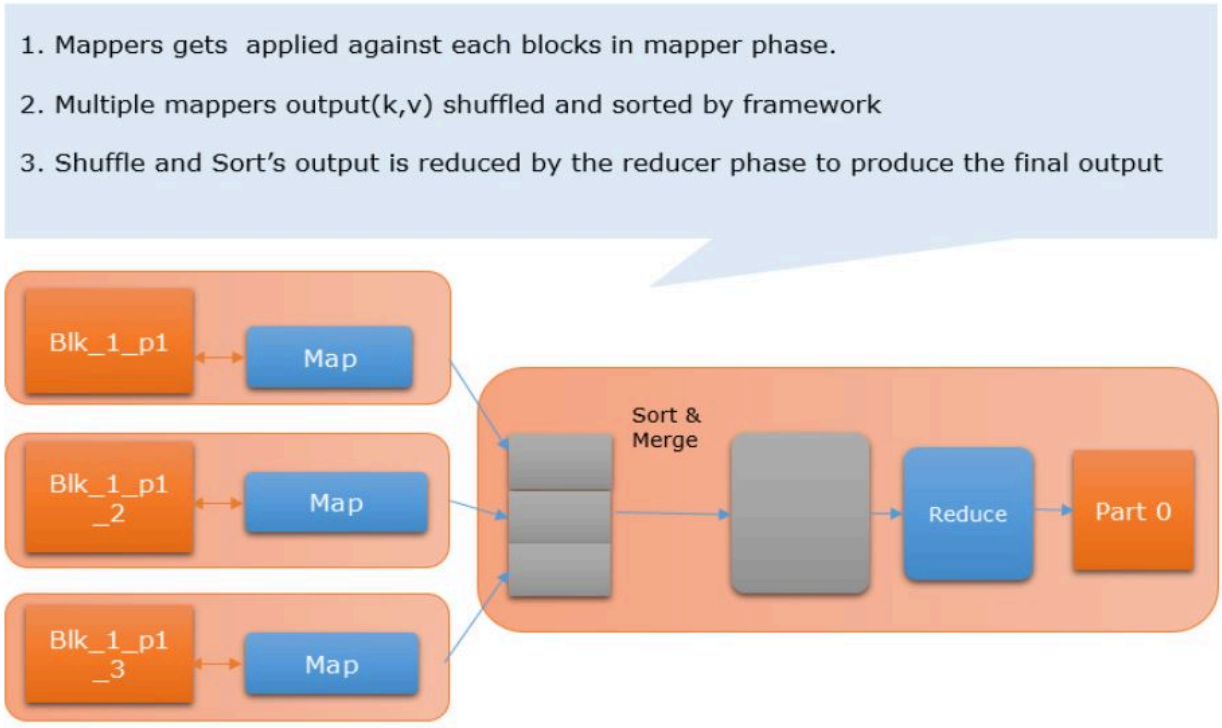
1. **Mapper**: holds the coding logic of mapper function and applies the computation logic to ‘n’ number of data blocks spread across data nodes. Accepts a (key, value) pair input where k -offset address, v - entire record.

2. **Shuffle and Sort**: gets the (k,v) output from the Mapper phase. Duplicate values are removed and the various values of a single key get grouped based on the key name. The output of this phase is again a key value pair i.e. key, array of values(k, v[]). Cannot be customized.

3. **Reducer**: aggregates the values against each key. Reducer consolidates multiple mapper’s output and computes the final output of the job ,writes it to a single output file in an output directory.

4. **Combiner** (optional phase): used for optimizing the performance in MapReduce jobs. Various outputs of the mappers are locally reduced on node level. For example, if 5 mapper outputs(k,v) come from a single node and if it contains duplicates, then they get combined (locally reduced), meaning values of 5 keys gets aggregated on machine level and thereby the output would be k, group of values(k,v[]). This makes the shuffle and sort work even quicker, thereby enabling additional performance in MapReduce job running.

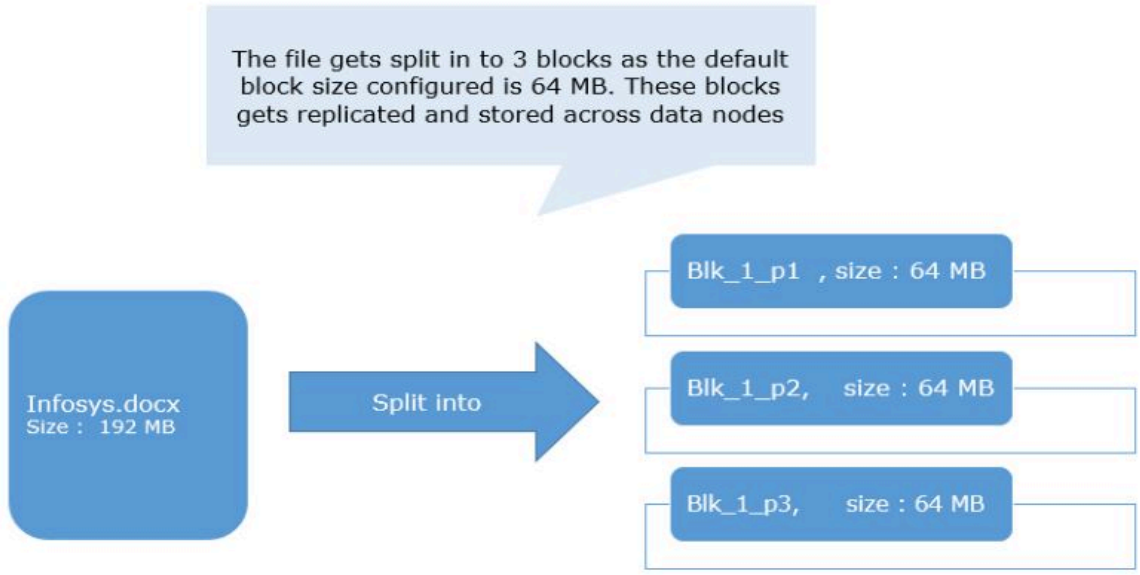




Load a local file into HDFS:

hadoop dfs –copyFromLocal "/localpath/Infosys.docx" "/hdfspath" (or)
hadoop dfs –put "/localpath/Infosys.docx" "/hdfspath"

Run a MapReduce jar against the dataset:
hadoop jar "MRjarfile" "/hdfspath/Infosys.docx" "hdfspath/outdirectory"



What happens if a node containing a few blocks fails in the middle of a MapReduce processing?

Speculative execution is an optimization technique in Hadoop.

Purpose: To handle slow or failed tasks efficiently.

- **Failure Handling:** If a mapper fails, the task is reassigned to another node with a replica of the data block.
- **Slow Task Handling:** If a mapper is slow, a new mapper task is assigned to a replica of the data block. The first task to complete is considered final; the other is terminated.

Benefits: Ensures faster overall job execution.

Mitigates the impact of slow or failed tasks.

The speed of a MapReduce job is determined by the slowest task, so speculative execution helps maintain optimal performance.

Example: word count analysis.

MAPREDUCE ARCHITECTURE

Daemon Services: Background services for running mapper and reducer logic, monitoring machine failures, and managing task failures.

Features : fault tolerance
parallel processing

2 core components as Daemon services (background services):

- Hadoop 1.x: Job Tracker and Task Tracker.
- Hadoop 2.x and above: Resource Manager and Node Manager.

Job Submission - Job Client: Submits MapReduce job using HDFS commands.

Hadoop 1.x daemons and architecture:

HDFS components:

- ➔ Name node:
  - ◆ master service
  - ◆ holds metadata (file splits, location, replication, etc..)
- ➔ Data node:
  - ◆ slave service
  - ◆ stores actual data blocks
  - ◆ provides replication for fault tolerance

MapReduce components:

- ➔ Job tracker:
  - ◆ master service
  - ◆ assigns tasks to task trackers
  - ◆ communication with name node
  - ◆ monitors task progress
- ➔ Task tracker:
  - ◆ slave service
  - ◆ runs on data nodes
  - ◆ executes mapper and reducer tasks
  - ◆ sends heartbeat notifications to Job tracker

Workflow of a MapReduce (MR) Job

1. Job Submission: Client submits a MapReduce Job to the Job Tracker.

   MR code is copied to a shared HDFS path.

2. Data Block Retrieval: Job Tracker contacts the Name Node.

   Retrieves locations of data blocks across various Data Nodes.

3. Task Scheduling: Job Tracker schedules mapper and reducer tasks.

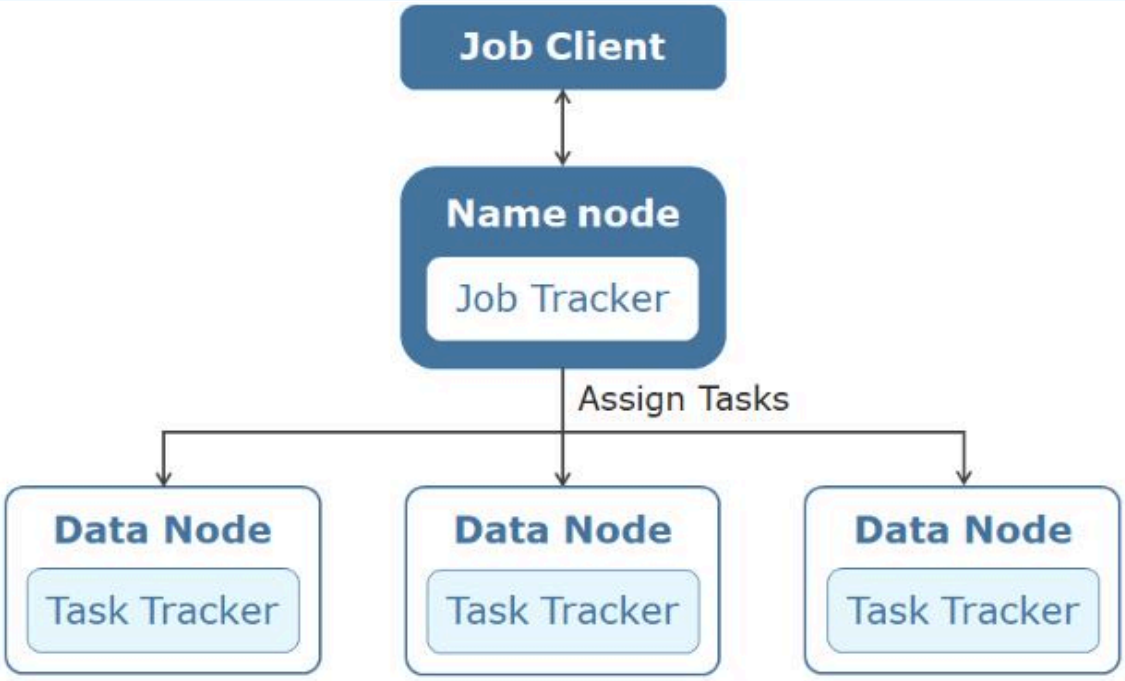
   Tasks are assigned to Task Trackers on different Data Nodes.

4. Mapper Execution:Task Tracker runs multiple mapper tasks against corresponding data blocks.

5. Reducer Execution: A single reducer task aggregates outputs from all mappers.

6. Output Generation: Analysis output is generated in the specified output directory.

7. Job Completion: Job Tracker prints a success status for the completed MapReduce Job.



Hadoop 2.x daemons and architecture:

Key components:

1. Resource Manager (RM):

Manages global resources	Centralized component
Scheduler allocates resources among running applications	Interacts with multiple AMs

2. Node Manager (NM) :

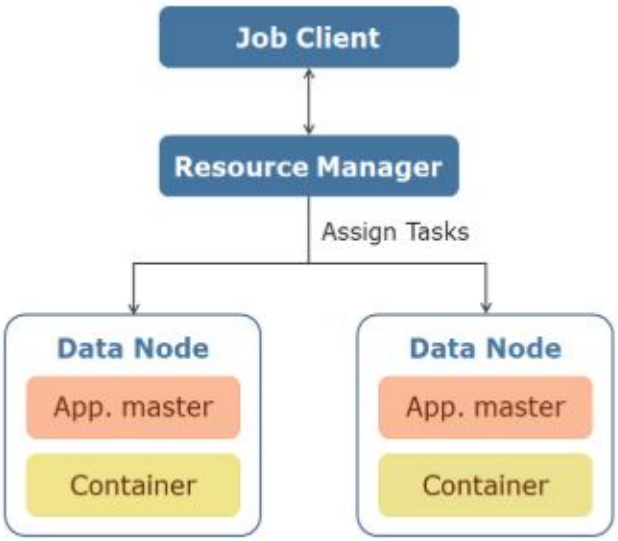
Runs on each cluster node	Distributed across nodes
Manages node level resources	Communicates with RM

3. Application Master (AM)

Framework specific instance for YARN jobs	Per application instance
Negotiates resources from RM	Manages job execution
Uses NM to execute tasks	

Workflow

1. Client Submission: Client submits a MR job to the RM's scheduler.
2. Task Scheduling: Scheduler assigns MR tasks to the AM of each data node.
3. Task Execution: AM negotiates and executes tasks in the machine's container.
4. Node Management: NMs send node status to RM. Manage resources, CPU, memory, and handle failures.



SERIALIZATION AND DESERIALIZATION (DeSe)

Serialization: process of converting objects to byte stream - converting structured data into its raw form (a format that can be easily stored or transmitted and then reconstructed later). It is required to transfer the object over the network i.e. for inter-process communication.
Deserialization: process of converting bytes back into programmatic objects. It is the reverse process of reconstructing structured forms from the data's raw bit stream form.

SeDe enables Hive to work with different file formats.

RPC (remote procedural call) serialization:

- Inter-process communication between nodes: implemented via RPCs;
- Serialization: converts object(mapper output) into byte stream, which is sent over the network.
- Deserialization: converts byte stream back into object. Data enters the reducer phase.

Why serialization ?
The intermediate data needs to travel between the various phases of MR. Data is serialized/deserialized at each phase.
Hadoop framework does this task for developers via a writable interface.

Writable and WritableComparable Interface

- Key/Value Pairs: Essential for data movement across nodes in a cluster.
- Serialization Format: MR framework requires a specific format for key/value serialization.

Writable Interface

- Purpose: Enables classes to function as **values** in the MapReduce framework.
- Methods:
    - void write(DataOutput out): Serializes the object to a byte stream.
    - void readFields(DataInput in): Deserializes the byte stream back into an object.

WritableComparable Interface

- Purpose: Enables classes to function as **keys** in the MapReduce framework.
- Inheritance: Extends the Writable interface.
- Additional Method: int compareTo(): Compares keys for sorting.

Characteristics

- Writable: Compact, fast, but not interoperable.
- Usage: Implemented by Hadoop classes like LongWritable, IntWritable, FloatWritable, etc.

INPUT OUTPUT FORMAT IN MR

Steps After Job Submission

1. Client calls getSplits(): Calculates the splits.
2. Send splits to JobTracker: JobTracker receives splits.
3. JobTracker identifies TaskTracker nodes: Based on splits, storage location.
4. Map Task calls createRecordReader: Uses InputFormat with the split as a parameter.

5. createRecordReader returns RecordReader: RecordReader acts as an iterator over records (key-value pairs).
6. Map Task Processes Records: Each record (key/value pair) is passed to the Map function.

Popular Input Formats for MapReduce

TextInputFormat: Default for plain text files.
Key: Byte offset address indicating the beginning of the line.
Value: Line content, excluding line terminators.

KeyValueTextInputFormat: Similar to TextInputFormat.
Key: First field before the tab character (/t).
Value: Remaining part of the line after the tab character.

SequenceFileInputFormat<K,V>: Reads sequence files containing binary data.
Key & value: User-defined, supports serialization and deserial.

Choosing the file format:

- Historical dataset with duplicates: Use TextInputFormat.
- Structured file with unique record identifier: Use KeyValueTextInputFormat.
- Binary sequence file: Use SequenceFileInputFormat.

Popular Output Formats for MapReduce

TextOutputFormat<K,V>: Writes records (key-value pairs) as lines of text. Default.
Key and value are written in String format with a tab between them.

SequenceFileOutputFormat<K,V>: Stores output in binary format.
Allows quick reading.

NullOutputFormat<K,V>: Outputs nothing.

12:30 pm

PERFORMANCE OPTIMIZATION IN MR

17.10.25, 9:50 am

MR jobs lead to latency on massive datasets.

Optimization parameters:

1. Combiner phase: combiner logic written inside a combiner function of a combiner class just similar to mapper and reducer classes. Combiners get configured in the Driver code (main function) to improve the performance of Shuffle and Sort. This in turn optimizes the entire MR job itself. Mapper class's output is given as input to combiner class, which processes it and sends it to the shuffle and sort phase. Works only for commutative and associative reducer functions.

Without combiner:
Mapper 1 -> (cuba,1),(Mexico,1),(US,1),(US,1) goes to shuffle and sort
Mapper 2 -> (cuba,1),(Mexico,1),(US,1),(US,1) goes to shuffle and sort
Shuffle and sort-> (cuba,[1,1]),(Mexico,[1,1]),(US,[1,1,1,1])->Reducer->(cuba,[2]), (Mexico,[2]), (US,[4])

With combiner:
Mapper 1 -> (cuba,1),(Mexico,1),(US,1),(US,1) -> Combiner1 -> (cuba,1),(Mexico,1),(US,2)
Mapper 2 -> (cuba,1),(Mexico,1),(US,1),(US,1) -> Combiner2 -> (cuba,1),(Mexico,1),(US,2)
Shuffle and sort -> (cuba,[1,1]), (Mexico,[1,1]), (US,[2,2]) -> Reducer -> (cuba,[2]), (Mexico,[2]), (US,[4])

2. Partitioner: partitioning the mapper output k,v pairs with respect to k and grouping values with respect to that key. The default partitioner is **HashPartitioner** which controls the partitioning of keys from mapper output. It computes a hash value for the key from mapper output, and assigns the partition based on this result.
   Advantage- it ensures all records of the same key from mapper output are grouped together, all values of that same key are assigned to the same reducer. This allows even distribution of mapper outputs to reducer. The number of reducers is equal to the number of partitions.
   Issues- may also lead to performance issues if not handled properly. Few reducers hold more values for a partition key, compared to few other reducers holding very lesser partition values.
   Solution- custom practitioners.
   Example- Reduce (salary >=50k, salary<50k)

3. Compression algorithm: Input files arriving into HDFS shall be compressed before storage to get space benefits. Input files which are compressed gets decompressed automatically while getting read by a MapReduce program.

Popular compression formats:

Compression Format	Tool	Algo	File name Ext	Multiple Files
DEFLATE	N/A	DEFLATE	.deflate	No
gzip	gzip	DEFLATE	.gz	No
ZIP	zip	DEFLATE	.zip	Yes
bzip2	bzip2	bzip2	.bz2	No
LZO	lzop	LZO	.lzo	No

Codec- implementing CompressionCodec interface. In Hadoop, Codec API classes are available for implementing compression and decompression algorithms.

Codec classes that can be configured in the driver code:

Compression Format	Hadoop Compression Codec
DEFLATE	org.apache.hadoop.io.compress.DefaultCodec
gzip	org.apache.hadoop.io.compress.GzipCodec
bzip2	org.apache.hadoop.io.compress.BZip2Codec
LZO	com.hadoop.compression.lzo.LzopCodec

MR JOINS

Allows 2 or more datasets to be joined, to get insights from multiple data sources.
While joining datasets, if one dataset's size is smaller than the other, the smaller dataset is distributed to every datanode in the cluster. Then mapper or reducer uses smaller dataset to perform lookup for matching records from bigger dataset, and combine both to produce output joined data records.

Types of Joins in MR:

1. Map side Join- Mapper uses a smaller dataset to perform lookups on a larger dataset.

   Process: Join is performed before data reaches the map function.

Input to mapper must be partitioned and sorted by keys.

2. Reduce side Join: Reducer uses a smaller dataset to perform lookups on a larger dataset.

Process: No need for structured or partitioned datasets.
Mapper emits join keys and tuples from both datasets.
Shuffle and Sort phase groups values by identical keys.
Reducer joins data records with the same keys.

Reduce side join is simpler and easier to implement due to the automatic shuffle and sort phase.

How Reduce side join works

1. Mapper Phase: Reads data and generates (K, V) pairs where K is the join key.
2. Shuffle and Sort Phase: Sorts and groups data by keys, sending identical keys to the same reducer.
3. Reducer Phase: Joins data records with identical keys.

BEST PRACTICES IN MR

Larger HDFS blocks: improves performance.

Use Combiner: for local aggregation.

Compression: for intermediate output.

Distributed Cache: use for smaller files only.

Map Only Jobs: set the number of reducers to zero.

Number of Reducers: slightly less than available reduce slots.
Number of reduce slots = Number of nodes * mapred.tasktracker.reducetasks.maximum

Mappers: process optimal amount of data.
smaller parts increase waiting time.
large blocks make re-execution expensive.

APACHE HIVE

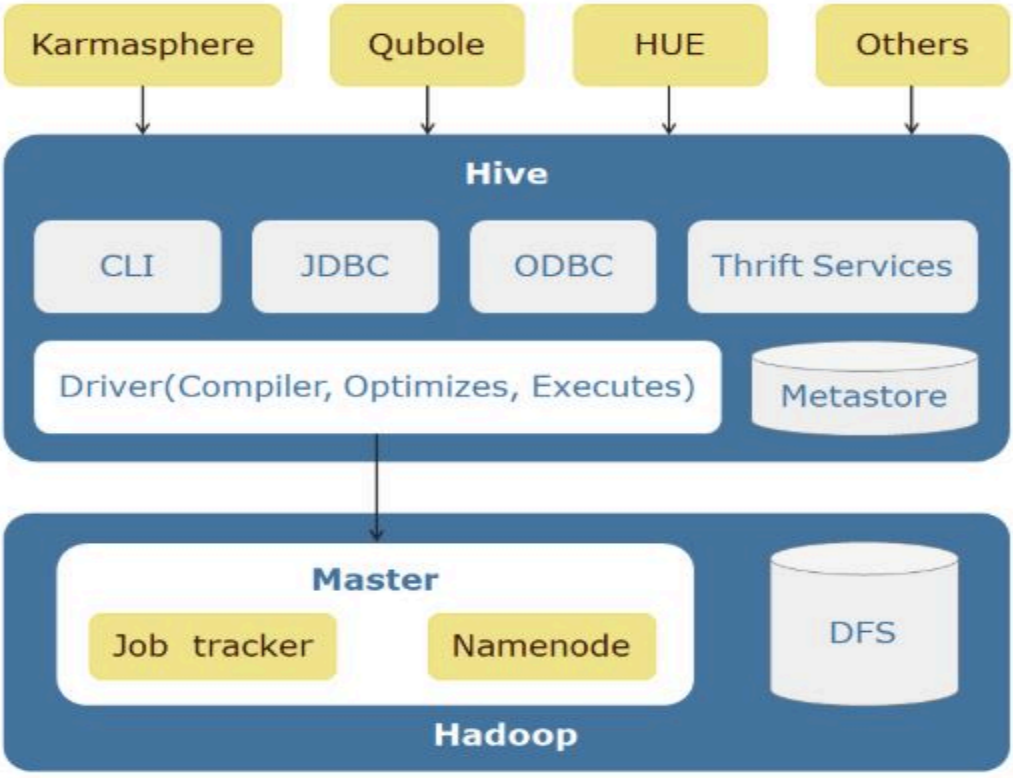
Hive is a data warehousing solution built on top of HDFS for querying structured and semi-structured data in HDFS. It is mainly used for data querying, analysis and summarization. It helps to improve the developer productivity which comes at the cost of increasing latency. Building blocks: Databases, tables, partitions and buckets.

Hive offers Hive Query Language (HiveQL) which is quite similar to SQL language in RDBMS. Hive queries are internally translated to MapReduce jobs that are executed on the Hadoop cluster.

Features of HiveQL:

- can query large volumes of data in a short time;
- supports primitive and complex data types to accommodate all types of data;
- supports joins, indexes and views like any other databases;
- enhances its query performance through partitioning and bucketing;
- provides custom functionalities with User Defined Functions (UDF);

HIVE ARCHITECTURE



User Interface

- HUE, Qubole, Karmasphere: Interfaces to access HDFS for analysis.
- Hive Web UI: Web-based interface.
- Hive Command Line: Command-line interface.

Metastore

- System catalog: Contains metadata about table schema and system schema.

- RDBMS: Implemented using MySQL.
- Metadata Storage: Stores schema, table, database, column data types, and HDFS mapping.

Driver

- Function: Compiles, executes, and optimizes Hive queries.

Thrift Server

- Purpose: Allows clients to access Hive using languages like C++, Java, Ruby, etc.

HIVE vs RDBMS

	HIVE Datawarehouse	RDBMS Database
Schema	Schema on read model	Schema on write model
Operation	Write once, read many times	Read and write many times
Scalability	Horizontally scalable; low cost infra	Vertically scalable; high cost
Data analysis	Good for static data analysis	Good for real time data analysis
Data ownership	Does not own data, as data is stored in HDFS	Owner of data and manages it
Computation	Supports parallel computation	Supports serial computation

HIVE DATATYPES

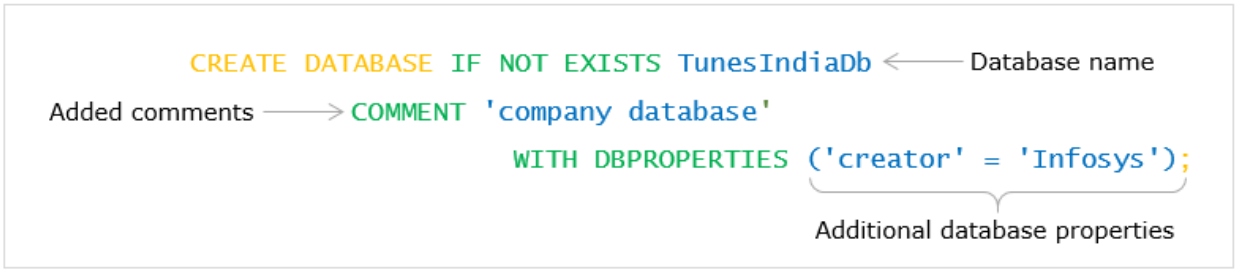
Primitive types: tinyint, smallint, int, bigint, boolean, string, timestamp, float, double, binary
Collection types:

array- to store ordered list of homogeneous data
struct - represents heterogeneous type of data
map - stores data as key value pairs with a value associated to each unique key

DESIGNING DATA MODEL

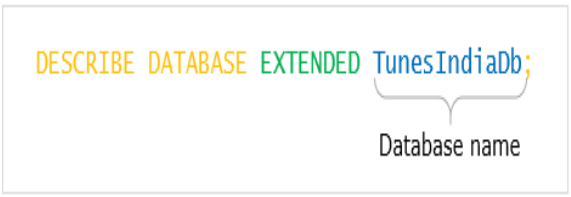
-WORKING WITH DATABASES

DB creation:



By default, a database in Hive is stored as a directory in HDFS with its name as <databasename>.db at the location /user/hive/warehouse/.

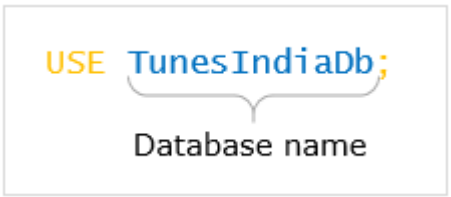
Describe: to view its location and properties.



DB listing: list all available DBs



Use DB:



Drop DB:



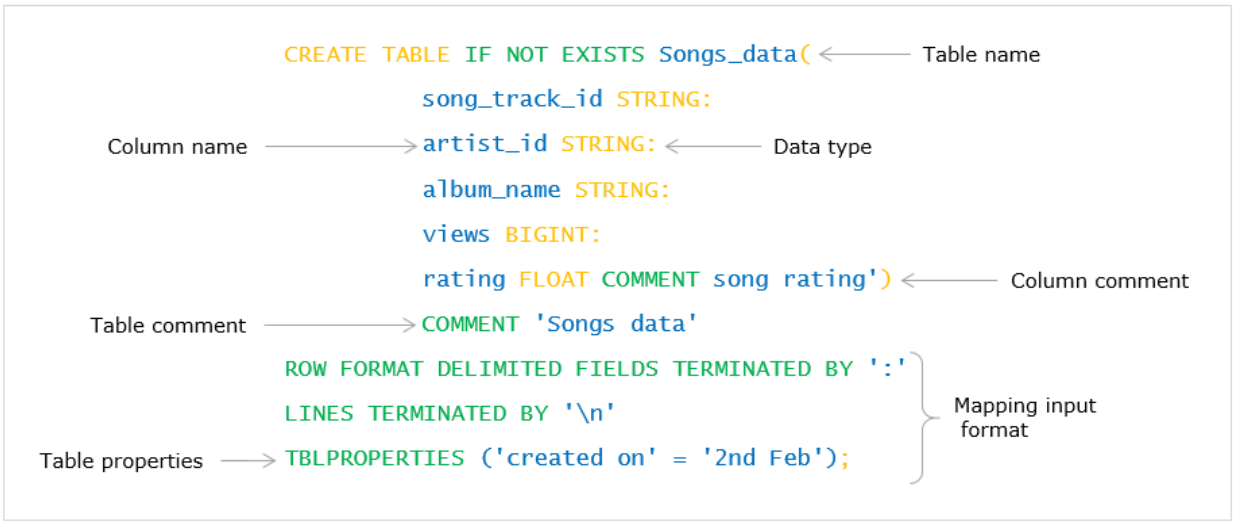
-WORKING WITH TABLES

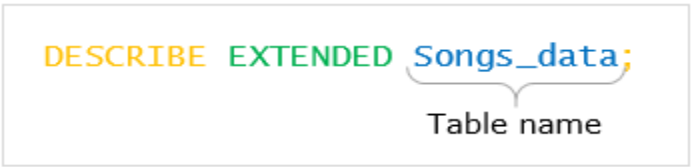
2 types

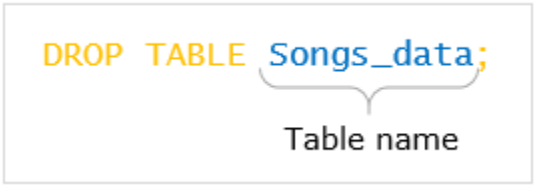
1. Managed / internal tables
    - life cycle of data in the table is controlled by Hive

- data is stored under the sub directory defined by 'hive.metastore.warehouse.dir'
- when table is dropped, data and metadata is deleted
- not a good choice for sharing data with other tools

2. External tables
   - Use the keyword EXTERNAL with CREATE TABLE
   - Life cycle of data in the table is NOT controlled by Hive
   - data is stored under the directory defined by LOCATION clause in the CREATE TABLE command
   - When table is dropped , data is not deleted but metadata is deleted
   - Better choice for sharing data with other tools







```
SHOW TABLES LIKE 'S*';
```

Regex expression

Display table names starting with S

```
LOAD DATA LOCAL INPATH '/home/hiveuser/infosys/songs_data.txt' INTO TABLE Songs_data;
```

File path — Table name

ALTER

Usage of ALTER statements	Description
`ALTER DATABASE TunesIndiaDb SET DBPROPERTIES('modified by' = 'Infosys');`	Add a new database property 'modified by'
`ALTER TABLE Songs_data SET TBLPROPERTIES ('created by' = 'Infosys');`	Add a new table property 'created by'
`ALTER TABLE Songs_data RENAME TO Songs_details;`	Rename the table 'Songs_data' to 'Songs_details'
`ALTER TABLE Songs_data CHANGE views song_views;`	Change column name from 'views' to 'song_views'
`ALTER TABLE Songs_data ADD COLUMNS ('best_comment' STRING);`	Add extra column 'best_comment' to Songs_data table

QUERYING AND SEARCHING MULTIPLE DATASETS

-HiveQL

SELECT

SELECT [ALL | DISTINCT] select_expr/field, select_expr/field, ...

FROM table [WHERE condition]

[GROUP BY col_list] [HAVING having_condition]

[CLUSTER BY col_list | [DISTRIBUTE BY col_list]

[SORT BY col_list]]

[LIMIT number];

JOINS

Used to combine records from two or more tables in the database by using values common to each one.

```
SELECT artist_name, song_track_id <---- Column data
FROM Artist_data JOIN Songs_data <---- Joining tables
ON Songs_data.artist_id = Artist_data.artist_id; <---- Joining condition
```

Outer joins: Left, Right, Full

```
SELECT song_track_id, artist_name <---- Column data
FROM Artist_data  FULL OUTER JOIN Songs_data <---- Joining tables
ON Songs_data.artist_id = Artist_data.artist_id <---- Joining condition
where (artist_name is NULL) OR (song_track_id is NULL); <---- Filter condition
```

INDEXES

Provides references to records in the table. Are created on frequently used columns for faster retrieval.

```
CREATE INDEX artist_indx <---- Index name
ON TABLE Artist_data(artist_name) <---- Column on which index is built
AS 'org.apache.hadoop.hive.ql.index.compact.CompactIndexHandler' <---- Index type
WITH DEFERRED REBUILD
IDXPROPERTIES ('creator'='Infosys'); <---- Index properties
```

'With deferred build' ensures that the new index starts empty irrespective of the data present in the table.

```
SHOW INDEX ON Artist_data;
              Table name
```

```
ALTER INDEX artist_indx ON Artist_data REBUILD;
```
Index name — Table name

```
DROP INDEX artist_indx ON Artist_data;
```
Index name — Table name

VIEWS

Virtual table. Materialized only when it is queried.

```
CREATE VIEW IF NOT EXISTS song_rating_view
AS SELECT * FROM Songs_data WHERE rating > 4 ;
```
View name — Select query

Can add TBLPROPERTIES and COMMENTS also.

APACHE SPARK

Spark is a computational tool in Hadoop stack which can replace MapReduce to perform high speed computation with lesser and easier lines of code.

Spark is a lightning fast, in-memory (RAM) computation tool to process big data files stored in Hadoop's HDFS, NoSQL DBs or on a local file system. It was introduced by Apache Software Foundation and has rich APIs available in Scala, Java and Python for data computation.

It is based on Hadoop's MapReduce and extends MapReduce model for quick computations. Unlike MapReduce, Spark processes intermediate data of computation in RAM, which is faster compared to MapReduce.

Spark holds processing data (programmatic objects) in disk and does computations when the RAM size is limited and still works 10 times faster than MapReduce.

RDD (resilient distributed dataset) is the fundamental data structure, serving as an immutable, distributed collection of objects partitioned across a cluster of machines.

The entire set of Spark statements reside inside a **Lineage graph** with every statement's RDD holding the pointer reference to its previous statement's RDD. Hence when an RDD is lost it gets recreated from its dependencies in Lineage graph.

SPARK DAG (directed acyclic graph)

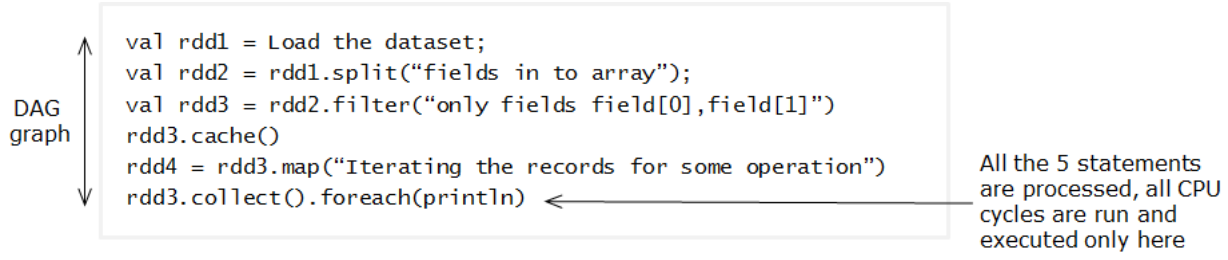
In Spark, any coding logic leads to a DAG creation. It holds the track of operations applied on RDD.

Spark DAG holds 2 kinds of statements:

- Transformation
- Action

Transformation Statements	Action Statements
Transformation statements are used for filtering and parsing functionalities	Action statements are used for aggregation
Many transformations appears with in a lineage graph and executes only at the last during an action	Usually one or two Action statements exists with in a lineage graph and each action evaluates all its proceeding transformation together
Result of every transformation leads to a new RDD creation with out evaluation in the lineage graph	Result of action statement is the final output of all RDD computations in the lineage graph
Eg. filter(), split()	Eg. count(), collect()

Every Spark Scala/Java/Python transformation statement leads to an RDD creation. Only transformation statements lead to RDD whereas action statements return some other data type.



- Four RDDs (rdd1,rdd2,rdd3,rdd4) get created within a DAG sequence based on the transformations applied.
- DAG helps to process data in a fault tolerant manner. When any RDD or a partition is lost in the middle of execution, it gets recreated.
- The entire graph gets executed only at the last line rdd3.collect().foreach(println) where an action is encountered.

All transformation statements get computed collectively, lazily, only during action statements, leading to high performance. This is called **Lazy evaluation**.

At the 6th statement the entire DAG gets evaluated and a logical plan gets created which is then converted to a physical plan and executed using **Spark workers** called **Executors**.

SPARK WORD COUNT DEMO

spark -shell : to start the Spark CLI

val rdd1 = sc.textFile("file:/home/datafile/demodatafile") : to create RDD

val rdd2 = rdd1.flatMap(_.split(" ")) : to extract each word

val rdd3 = rdd2.map(w=>(w,1)) : to get word count, assign each word to count 1

val rdd4 = rdd3.reduceByKey(_+_) : aggregate the counts

rdd4.collect : to view the output

rdd4.saveAsTextFile("file:/home/output_spark") : to save the output in a file

SPARK ECOSYSTEM

Enables developers to :

- use simple SQL like queries for data analysis
- stream real time data and process it
- write ML algorithms to automate data analytics tasks
- store and process graph data

